WHITECODE LABS • ENTERPRISE AI AUTOMATION

WhiteCode Meet Automation

Zero-Touch Google Meet Discussion Tracking, Multilingual Speech Transcription (मराठी & English), AI Project Classification & Automated Employee Tasksheet Logging System

DOCUMENT VERSION	TARGET ENVIRONMENT	AI ENGINE	GENERATED DATE
1.0.0 (Production)	Node.js / TypeScript / Chrome Extension	Google Gemini 3.6 Flash	October 2026

1 Executive Summary & System Overview

WhiteCode Meet Automation is an enterprise software platform designed to eliminate manual timesheet entry for engineering teams. In standard fast-paced development environments, team members spend hours in client syncs, code reviews, and architectural discussions across Google Meet, but often forget to log tasks, record inaccurate project codes, or fail to accurately track real attendance time.

Furthermore, team discussions frequently code-switch between **English and Marathi (मराठी / "Marglish")**. Standard speech tools fail to recognize Indian workplace dialect or transcribe engineering terms inaccurately. WhiteCode Meet Automation solves this by combining browser audio capture, Google Gemini 3.6 Flash multimodal transcription, meeting interval math, and automated corporate tasksheet generation.

Key Innovations:

- **Zero-Touch Attendance & True Duration:** Tracks precise join/leave timestamps. If an employee connects from multiple devices or disconnects/reconnects due to network issues, overlapping intervals are mathematically merged to eliminate duplicate time counts.
- **Multilingual Speech Transcription:** Transcribes spoken Marathi into Devanagari script and English technical terms verbatim without phonetic corruption.
- **Discussion Extraction:** Summarizes meetings into 2–5 concrete bullet items and translates discussion context into clean corporate English for company tasksheets.
- **Dual Dual-Direction Sync:** Integrates with Google Calendar Webhooks and pushes entries to WhiteCode's REST API.

2 Keys, Credentials & Environment Variables

The system utilizes three primary security and configuration layers:

Credential / Key	Location	Sensitivity	Purpose & Usage
Google OAuth Web Client `client_id`, `client_secret`	`credentials.json` (Project Root)	**High (Confidential)**	Issued by Google Cloud Console. Authenticates the backend to call Google Calendar API and Google Meet API on behalf of the organization.
Google OAuth Tokens `access_token`, `refresh_token`	`token.json` (Project Root)	**High (Confidential)**	Holds active OAuth session. Automatically generated during user consent. The backend automatically saves refreshed tokens here via `oauth2Client.on("tokens")` to prevent session expiration.
`GEMINI_API_KEY`	`.env`	**High (Confidential)**	Google AI Studio key. Authorizes calls to **Gemini 3.6 Flash** via the Google AI Files API for audio transcription and discussion classification.
`PUBLIC_WEBHOOK_URL`	`.env`	Medium	Public HTTPS endpoint (Cloudflare Tunnel or ngrok) that Google Calendar sends push webhook notifications to whenever a meeting is created or updated.
`EXTERNAL_SYNC_ENABLED`	`.env`	Low	Boolean toggle (`true` / `false`) enabling or disabling real-time sync with the company HR/Tasksheet REST portal.
`EXTERNAL_TASKSHEET_API_KEY`	`.env`	**High (Confidential)**	Bearer token or API key for authenticating with WhiteCode's central HR REST API.

3 Complete End-to-End System Flow

1. **Meeting Trigger & Discovery (Dual-Ingestion)**
   Meetings are detected either via Google Calendar Webhook (scheduled calls) or directly through the Chrome Extension when an impromptu call begins on `meet.google.com`.

2. **Audio & Live Caption Capture (Chrome Extension)**
   The Chrome Extension records high-fidelity microphone audio (128kbps Opus in WebM container) with noise suppression while simultaneously observing DOM live captions, filtering out Google Meet system toasts.

3. **Speech-to-Text via Gemini 3.6 Flash**
   Audio chunks are uploaded to the Google AI Files API. Gemini 3.6 Flash transcribes verbatim Marathi and English speech, preserving technical keywords (API, SQLite, UI, testing, etc.).

4. **AI Discussion Topic Extraction & Project Classification**
   Gemini analyzes the full transcript alongside the active project catalog. It extracts 2–5 concrete discussion topics, assigns the project ID, and creates formal corporate task descriptions.

5. **Duration Mathematical Consolidation & Disambiguation**
   The duration engine sorts all participant timestamps and merges overlapping reconnect intervals. Disambiguation algorithms match participants (e.g. distinguishing Suraj Dantrao vs Suraj WhiteCode).

6. **Tasksheet Generation & External API Push**
   Entries are saved to SQLite under each employee's individual tasksheet with formatted effort (e.g. "1 hr 30 mins"). If external sync is enabled, entries are pushed via REST API to the WhiteCode HR portal.

4 Backend Architecture & Logic Breakdown

4.1 Google Calendar Webhook Handler

When Google Calendar detects meeting changes, it dispatches an HTTPS POST notification to `/google/calendar/webhook` with headers `x-goog-channel-id` and `x-goog-resource-id`. The backend verifies the channel against `calendar_watch_channels`, returns `200 OK` immediately, and launches asynchronous event synchronization.

4.2 Overlapping Interval Duration Engine (`duration.service.ts`)

Network disconnects or multi-device joins cause participants to generate fragmented sessions. The backend merges intervals:

```
// Sort intervals chronologically by join time
intervals.sort((a, b) ⇒ a.start - b.start);
let merged = [intervals[0]];

for (let i = 1; i < intervals.length; i++) {
  let prev = merged[merged.length - 1];
  let curr = intervals[i];
  if (curr.start ⩽ prev.end) {
    prev.end = Math.max(prev.end, curr.end); // Merge overlapping window
  } else {
    merged.push(curr);
  }
}
// Sum durations of distinct merged intervals
```

4.3 Heuristic Employee Matching (`tasksheet.service.ts`)

To resolve participants to employee records without errors:

- **Tier 0:** Exact match in `employee_email_aliases` table.
- **Tier 1:** Exact lowercase match on official company email.
- **Tier 2:** Token-level discriminator scoring. Matches on distinctive full names or numeric IDs award +2,000 points; mismatched tokens subtract -1,000 points.

5 REST API Specification

Method	Endpoint	Request Payload / Params	Description
GET	`/`	None	Dashboard home; checks OAuth tokens and displays quick links.
GET	`/oauth2callback`	`code`, `scope` query parameters	Google OAuth redirect handler; persists refreshed tokens to `token.json`.
POST	`/google/calendar/webhook`	Headers: `x-goog-channel-id`, `x-goog-resource-id`	Google Calendar push webhook receiver.
POST	`/api/meet/audio-chunk`	`multipart/form-data`: `audio` (WebM), `meetingCode`, `speaker`, `participants`	Receives meeting audio, uploads to Gemini Files API, transcribes speech, and auto-updates tasksheet.
POST	`/api/meet/caption-stream`	`{ meetingCode, speaker, text }`	Appends real-time live captions scraped from Meet DOM.
GET	`/api/meet/:code/transcript`	Path parameter: `meetingCode`	Returns compiled real-time transcript for the meeting.
POST	`/api/meet/finalize-discussion`	`{ meetingCode, participants }`	Invokes Gemini topic extraction and creates tasksheet entries.
GET	`/tasksheets`	Query: `employeeId`, `date`, `projectId`	Visual dashboard to inspect, filter, and edit employee tasksheets.
POST	`/api/sync/external`	None	Triggers bidirectional sync with WhiteCode's HR/Tasksheet REST API.
POST	`/api/aliases`	`{ employeeId, aliasEmail, aliasName }`	Maps secondary email addresses to an official employee record.

6 Chrome Extension Architecture

The Chrome Extension is built on **Manifest V3** and operates inside `meet.google.com`.

A **Content Script (`content.js`)**

Injected automatically into Meet calls. Captures microphone audio using `navigator.mediaDevices.getUserMedia` with 48kHz sampling and Opus encoding. In parallel, a `MutationObserver` scrapes live captions while filtering out system messages like "Microphone is muted" or "X has left the call".

B **Popup Controller (`popup.html` / `popup.js`)**

Provides a dark-themed UI displaying server connection health, meeting code pill, live recording indicator, real-time scrolling transcript box, and the "Finalize & Fill Tasksheet" button.

C **Background Service Worker (`background.js`)**

Manages extension lifecycle, badge counts, and inter-tab communication.

7 Database Schema (SQLite: `meetings.db`)

Table Name	Key Columns	Description
`meetings`	`id` (PK), `google_event_id`, `title`, `date`, `start_time`, `end_time`, `duration_minutes`, `meet_link`, `status`, `project_id`, `project_name`, `transcript`	Primary registry of all calendar and instant meetings tracked by the system.
`meeting_transcripts`	`id` (PK), `meeting_code`, `speaker`, `text`, `source` (AUDIO / LIVE_CAPTION), `created_at`	Incremental store of verbatim speech fragments. Indexed on `meeting_code`.
`meeting_participants`	`id` (PK), `meeting_id` (FK), `participant_id`, `employee_email`, `display_name`, `total_duration_minutes`	Consolidated attendance record for each participant in a meeting.
`employees`	`id` (PK), `employee_id` (UK), `name`, `email` (UK), `active`	Internal WhiteCode employee directory.
`projects`	`id` (PK), `project_id` (UK), `project_name`, `description`, `active`	Catalog of company projects available for AI classification.
`employee_email_aliases`	`id` (PK), `employee_id` (FK), `alias_email` (UK), `alias_name`	Maps personal Gmail or alternate addresses to formal employee records.
`tasksheet_entries`	`id` (PK), `employee_id` (FK), `meeting_id` (FK), `project_id`, `date`, `task_category`, `task_details`, `effort_minutes`, `comments`, `sync_status`	The generated timesheet rows. Contains full discussion topic bullets and duration.
`calendar_watch_channels`	`id` (PK), `channel_id` (UK), `resource_id`, `calendar_id`, `expiration`	Active Google Calendar push notification webhook channels.

8 Quick Setup & Operational Guide

Step-by-Step Setup:

1. **Install dependencies:** Run `npm install` in the project directory.
2. **Configure Credentials:** Ensure `credentials.json` is placed in the project root and `GEMINI_API_KEY` is set in `.env`.
3. **Start Backend:** Run `npm run start` (server starts on `http://localhost:3000`).
4. **Authorize Google:** Open `http://localhost:3000/` in a browser and click "Grant Google Meet Permissions".

5. **Load Chrome Extension:** Navigate to `chrome://extensions`, enable Developer Mode, click "Load unpacked", and select the `chrome-extension` folder.
6. **Verify:** Join a Google Meet call, open the extension, click "Start Discussion Tracking", speak in English or Marathi, and click "Finalize & Fill Tasksheet". Open `http://localhost:3000/tasksheets` to see the populated entries.
